レポート:オンライン対戦格闘ゲームの開発

学籍番号: 25M38019 **氏名**: 吉田和真

1. 開発背景と目的

本プロジェクトでは、授業で学んだ分散システム、マイクロサービス、クラウドコンピューティングの概念を応用し、実際のアプリケーション開発を通じてその複雑性とメリットを理解することを目的としている。

このシステムが必要な背景:従来のモノリシックなシステムが抱えるスケーラビリティ、可用性、開発の柔軟性といった課題を解決するため、認証、マッチング、ゲームセッション管理を独立させたマイクロサービスアーキテクチャを採用し、オンライン対戦格闘ゲームの基盤を構築した。

2. システムの設計と構成

2.1. ファイル構造と各コンポーネントの役割

本システムは、3 つの論理的な層(クライアント、設定、サービス)と 7 つの Docker コンテナで構成される。

```
distributed_game_project
├── client                # クライアント層: ブラウザ上で動作するフロントエンドコード
│   ├── game_client.js  # 主要ロジック、API 通信、WebSocket 接続を管理
│   ├── index.html      # アプリケーションのエントリポイント
│   └── style.css       # クライアントのスタイル定義
├── server_config       # 設定層: Nginx 設定ファイルと Docker Compose 定義
│   ├── S1_docker-compose.yml # サービス間の接続と環境変数設定 (S1 側)
│   ├── S1_nginx.conf   # S1 サーバーのリバースプロキシと静的ファイル配信設定
│   ├── S2_docker-compose.yml # サービス間の接続と環境変数設定 (S2 側)
│   ├── S2_nginx.conf   # S2 サーバーのリバースプロキシと静的ファイル配信設定
│   ... (db_data, db_init_scripts など)
└── services            # サービス層: バックエンドのマイクロサービス群
    ├── approval_service      # ユーザー登録申請の承認・却下フローを管理
    ├── auth_match_service    # ユーザー認証とマッチングのビジネスロジックを実行
    ├── data_service          # データベースへのアクセスを一元管理する API サービス
    └── game_session_service  # ゲームのリアルタイムロジック、入力処理を実行
```

2.2. 分散システムとしての特徴（マルチサービス構成）

本システムは、以下の 3 つの主要なアプリケーションサービスが連携して動作する。

1. **承認サービス (approval_service):** ユーザーの登録要求を受け付け、管理者の承認状態（pending, approved, rejected）を管理する。
2. **認証・マッチングサービス (auth_match_service):** ユーザー名とパスワードを検証し、認証済みユーザーに対して対戦モードに基づいたマッチングキューを管理する。
3. **ゲームセッションサービス (game_session_service):** マッチング成立後、WebSocket を介してクライアントと接続し、低遅延なゲームロジック（入力同期、衝突判定など）を実行する。
4. **データサービス (data_service):** 他のサービスからのデータアクセス要求に応える。

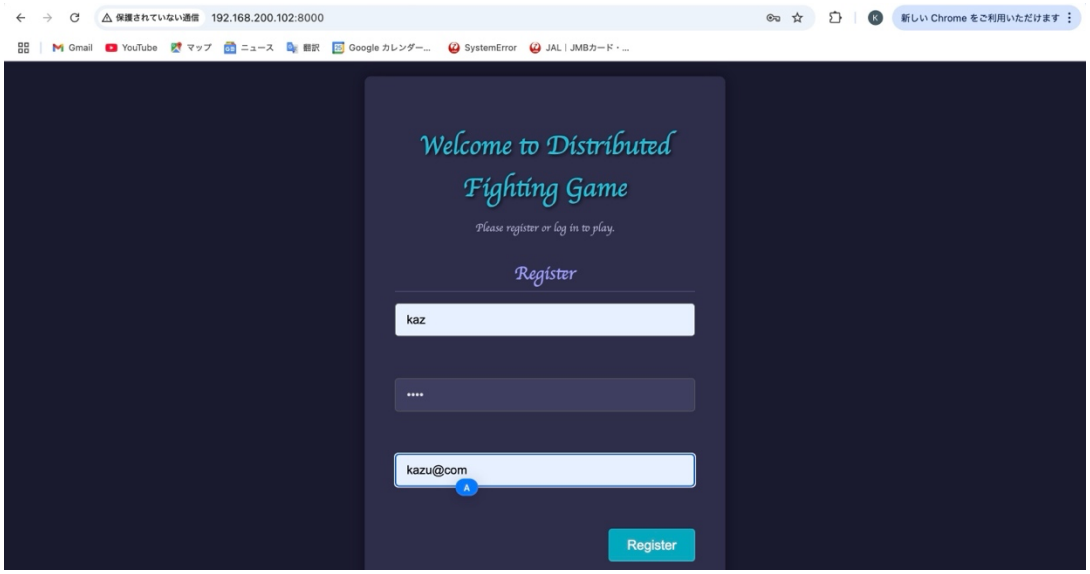
2.2.1. 承認サービス (approval_service)

目的: ユーザーの申請登録と、管理者の承認システムにより利用者を限定する。**サーバー2 が担当。**

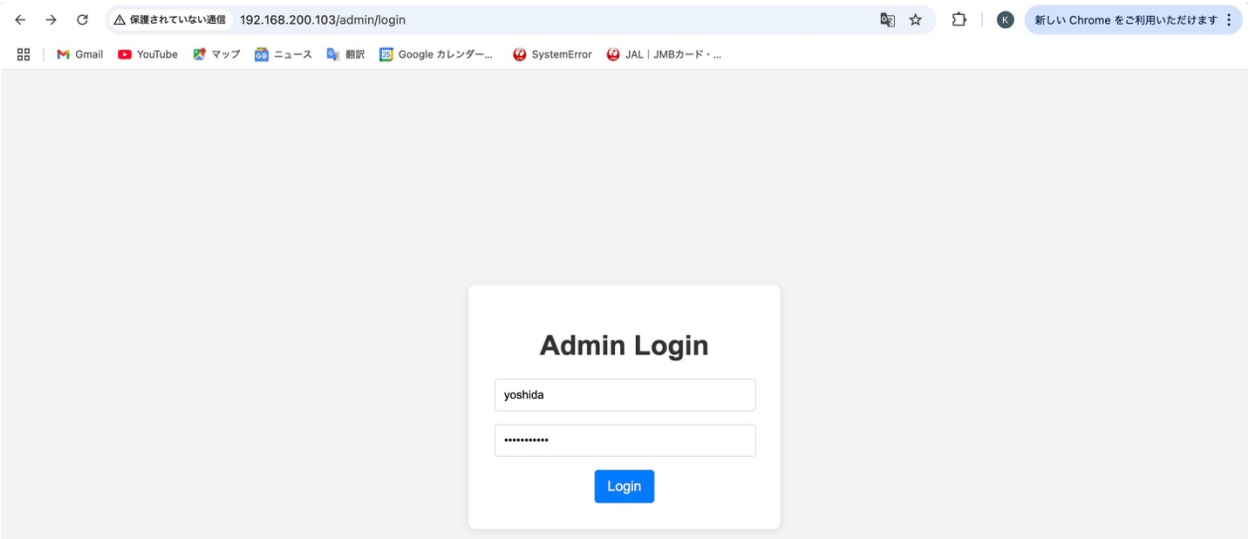
役割**:**

1. **申請受付 (/apply):** 新規ユーザーの申請を受け付け、データベースの users テーブルのステータスを pending（申請中）に設定する。
2. **ステータス管理:** ユーザーの承認状態（pending, approved, rejected）を管理し、ユーザー認証サービス auth_match_service からの問い合わせに応答する。
3. **管理者インターフェース:** 管理者が未承認ユーザーのリストを確認し、手動で承認/却下操作を行うための Web インターフェース（admin.html）を提供する。

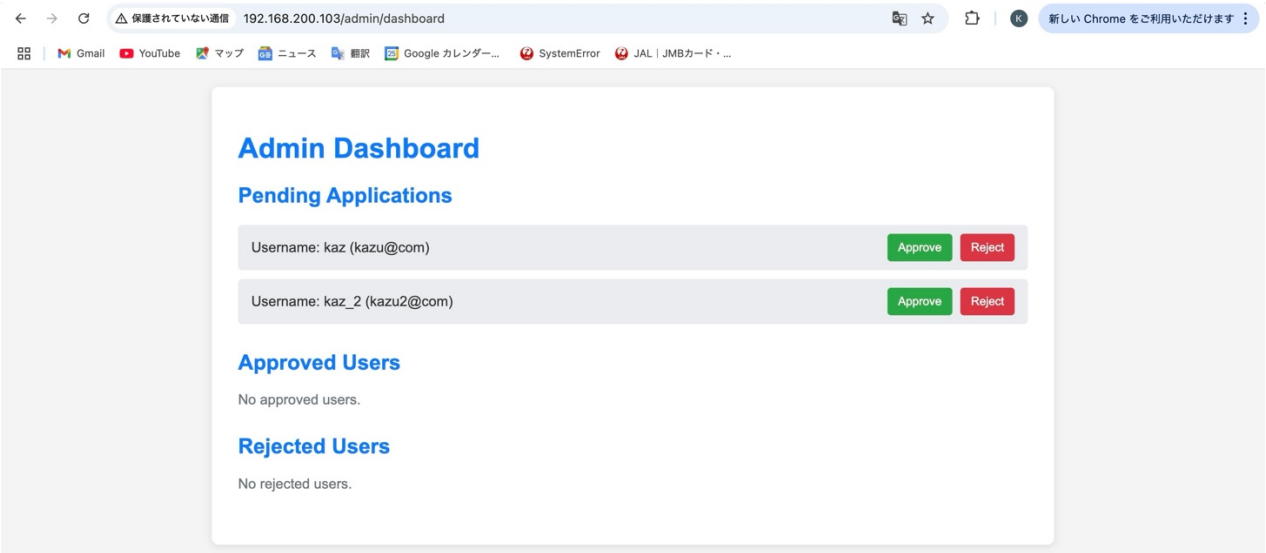
[ユーザー申請画面]※サーバー1



[管理者ログイン画面]※管理者画面は全てサーバー2 が担当

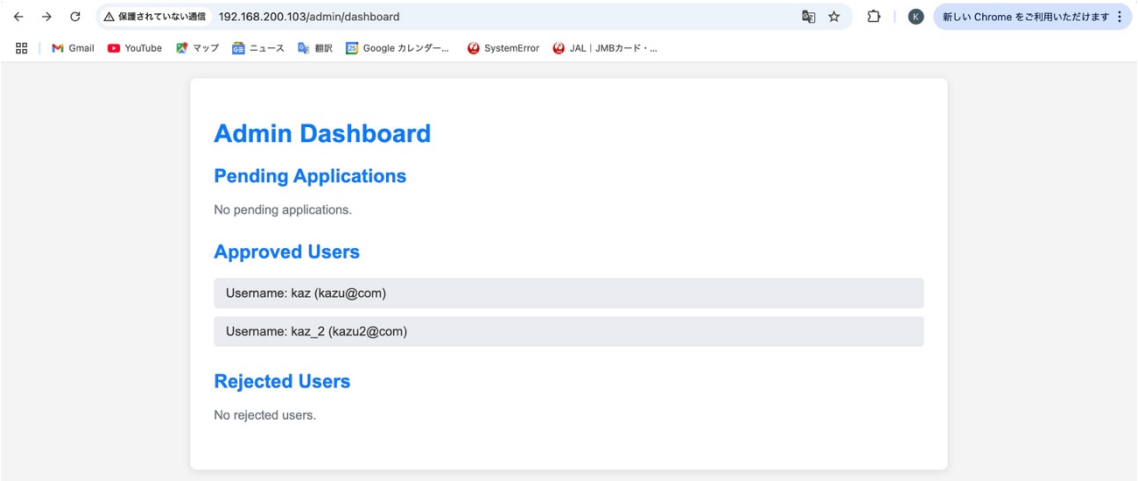


[管理者画面(ログイン後)]



[承認後直後の画面]

[承認後の管理者画面]



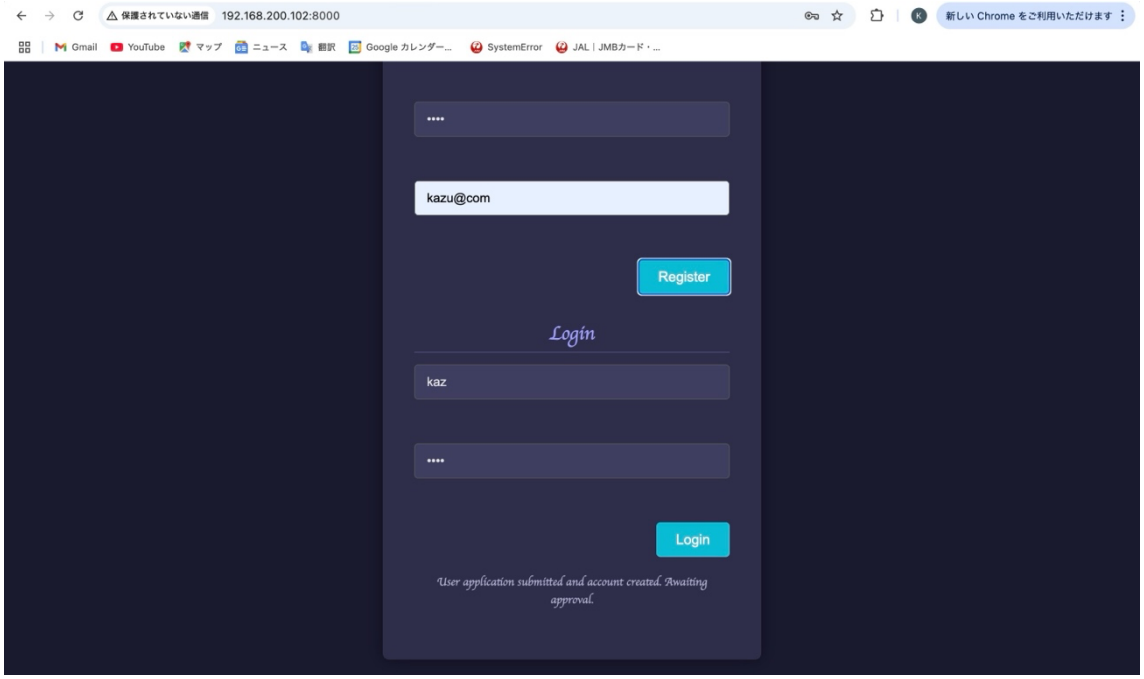
2.2.2. 認証・マッチングサービス (auth_match_service)

目的: ユーザーの本人確認(認証)と、対戦を希望するユーザー同士の組み合せ(マッチング)を実行し、ゲームセッションへの接続を仲介する。**サーバ 1 が担当。**

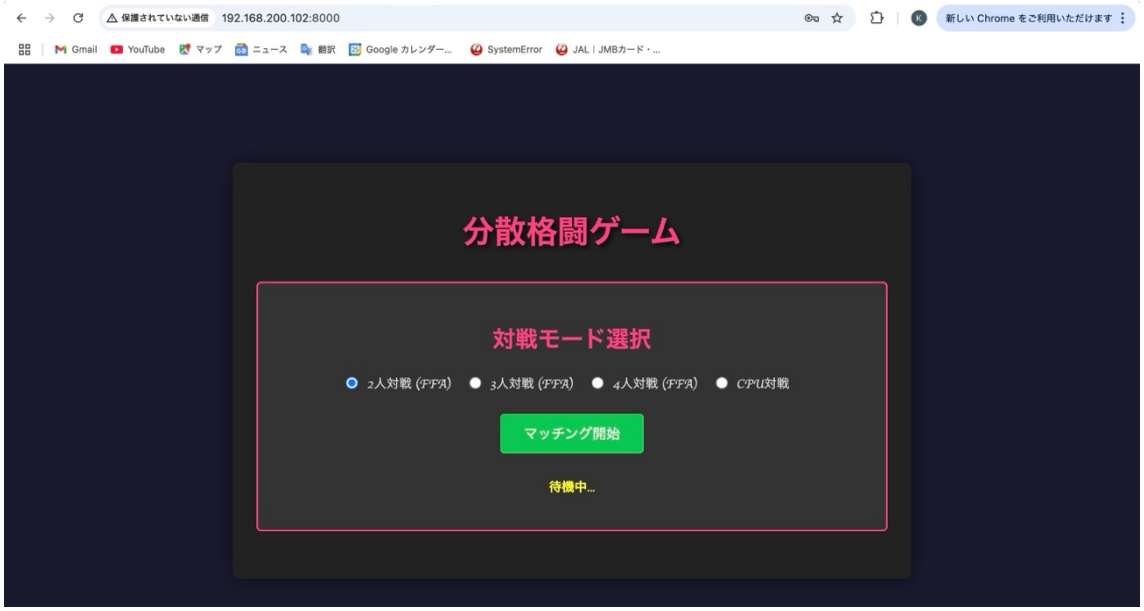
役割:

1. **認証 (`/login`):** ユーザー名とパスワードのハッシュをデータベースと照合し、本人確認を行う。成功後、承認サービス `approval_service` に承認ステータスを確認する。
2. **マッチング (`/match`):** クライアントから要求されたモード(例: `ffa_2p`)に基づき、ユーザーをキューに入れ、必要な人数が揃うとマッチングを成立させる。
3. **サーバー割り当て:** `config.py` で定義された負荷情報(SERVER_STATUS)に基づき、最も負荷の低い `game_session_service` の IP とポートを割り当てる。

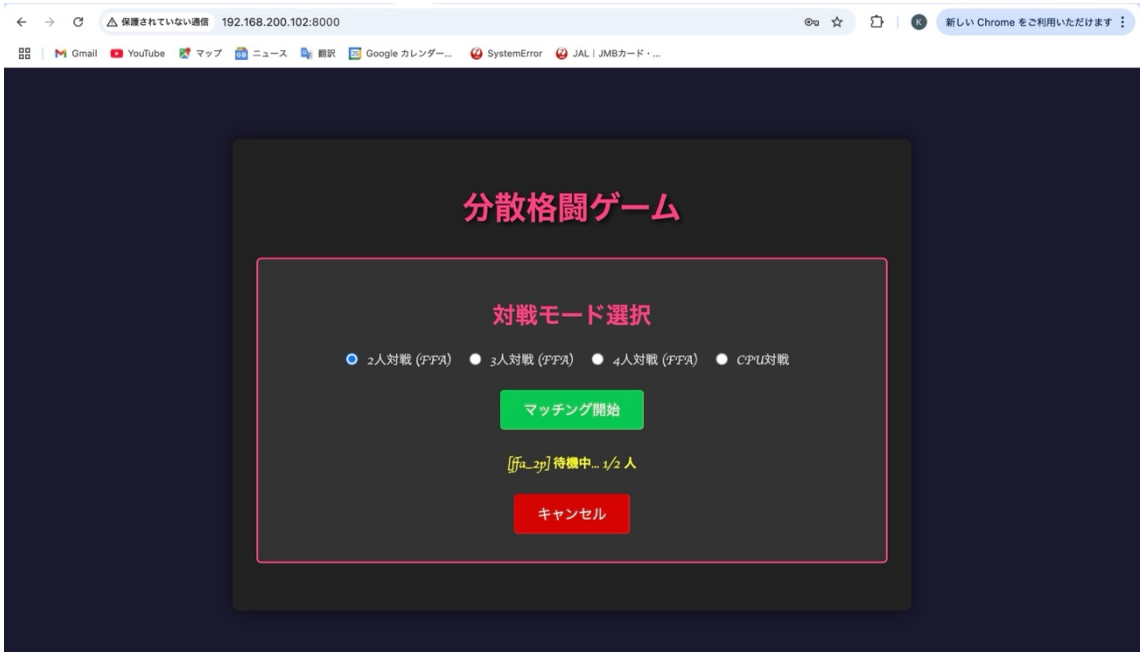
[ログイン画面]: 管理者による承認後ログイン可能



[マッチング画面(ログイン後)]※現在は 2 人対戦しか出来ない



[マッチング画面(モード選択後，待機中)]



[config.py]

```
# S1 と S2 のゲームセッションサービスのエンドポイントリスト
GAME_SESSION_ENDPOINTS = [
    {"ip": "192.168.200.102", "port": 8081, "server_id": "S1"},  # S1 のゲームセッション
    {"ip": "192.168.200.103", "port": 8082, "server_id": "S2"},  # S2 のゲームセッション
]
```

```python
# 現在の負荷(使用状況)を追跡する(セッション割り当てに使用)
SERVER_STATUS = [ep.copy() for ep in GAME_SESSION_ENDPOINTS]
for status in SERVER_STATUS:
    # アクティブなセッション数を格納
    status["active_sessions"] = 0
# 対戦モードの定義
MODE_CONFIG = {
    # 全員敵同士の対人戦 (FFA: Free-for-All)
    "ffa_4p": {"min_players": 4, "max_players": 4, "is_pvp": True}, # 4 人対戦
    "ffa_3p": {"min_players": 3, "max_players": 3, "is_pvp": True}, # 3 人対戦
    "ffa_2p": {"min_players": 2, "max_players": 2, "is_pvp": True}, # 2 人対戦
    # CPU 対戦モード
    "cpu": {"min_players": 1, "max_players": 1, "is_pvp": False}, # ユーザー1 人 + CPU
}
```

2.2.3. ゲームセッションサービス (game_session_service)

目的: マッチングが成立したプレイヤー間のリアルタイム通信を確立し、ゲームの進行、入力処理、状態管理といった核となるロジックを実行する。**サーバー1 が担当**(将来的にはロードバランサーを用いてゲームセッションごとに S1 と S2 に分けたい)。

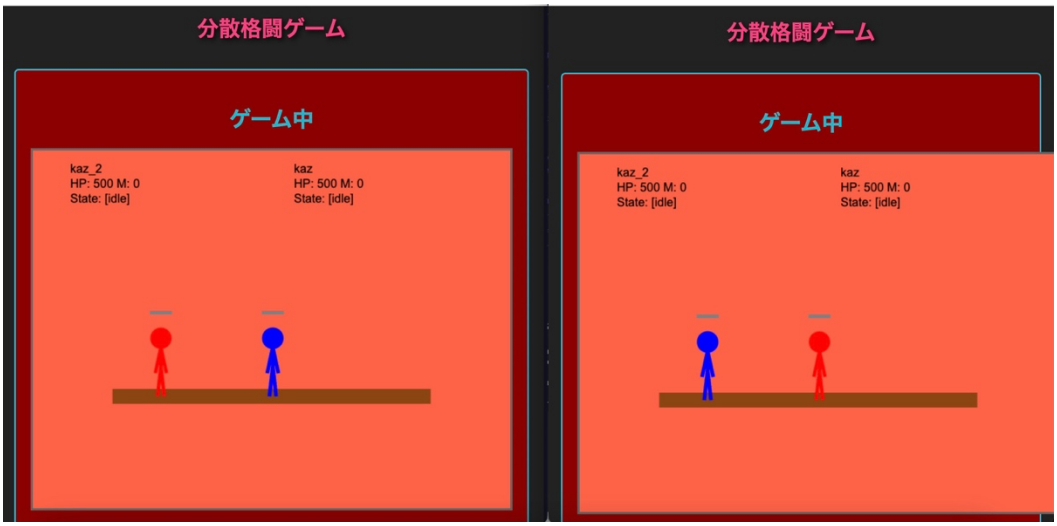
役割:

1. **WebSocket セッション管理:** クライアントからの WebSocket 接続を受け付け、ゲームセッションを立ち上げる。クライアントは本サービスの WebSocket エンドポイントに直接接続する。
2. **入力同期:** 各プレイヤーのキー入力情報を受信し、ゲームフレームに基づき、他のプレイヤーにブロードキャスト(同期)する。キー操作のたびに JSON メッセージを送信し、他のプレイヤーの状態を JSON メッセージで受信する。
3. **ゲームロジック:** キャラクターの位置計算、衝突判定、攻撃判定、ゲージの増減、HP 管理といった格闘ゲームのコアなロジック(`game_logic.py`)を実行する。

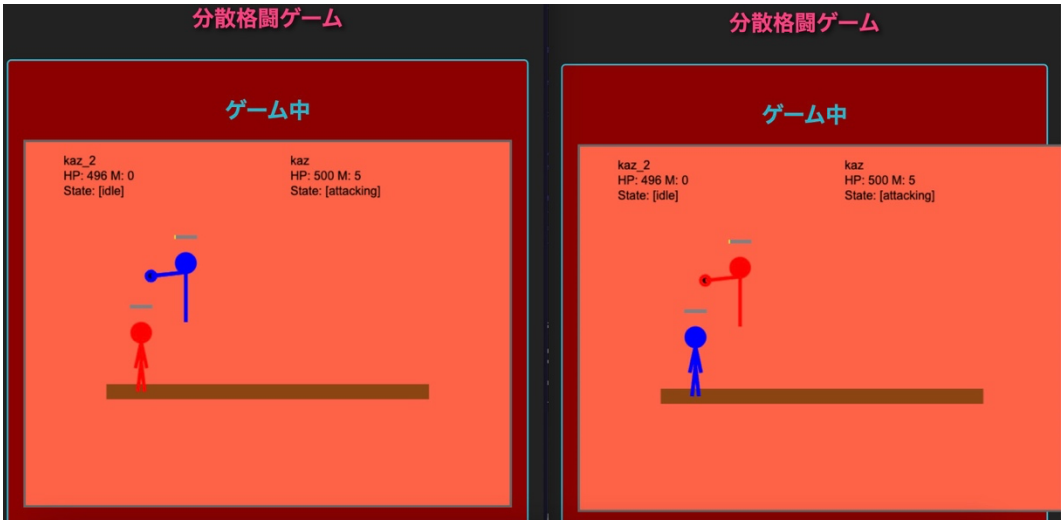
ルール:

青が自分、赤が相手、HP500 でゲームスタート。基本的な動きは走る(←, →)、ジャンプ(space key)のみ。攻撃モーションは 3 つで上(x キー)、下(s キー)、横攻撃(x キー)の 3 つがあり、それぞれ攻撃適応範囲、基礎攻撃力が違うが、相手との距離が近ければ攻撃力が高いという点では共通である。さらに攻撃が当たる度にゲージが 5 ずつ溜まっていき、100 になると必殺技(c キー)を使える。HP が 0 になるか、横に行きすぎて、あるいは下に落ちて画面外に出ると負ける。ただし、台から落ちてもジャンプで復帰するロジックを実装した。さらに攻撃したら反発するロジックも実装した。

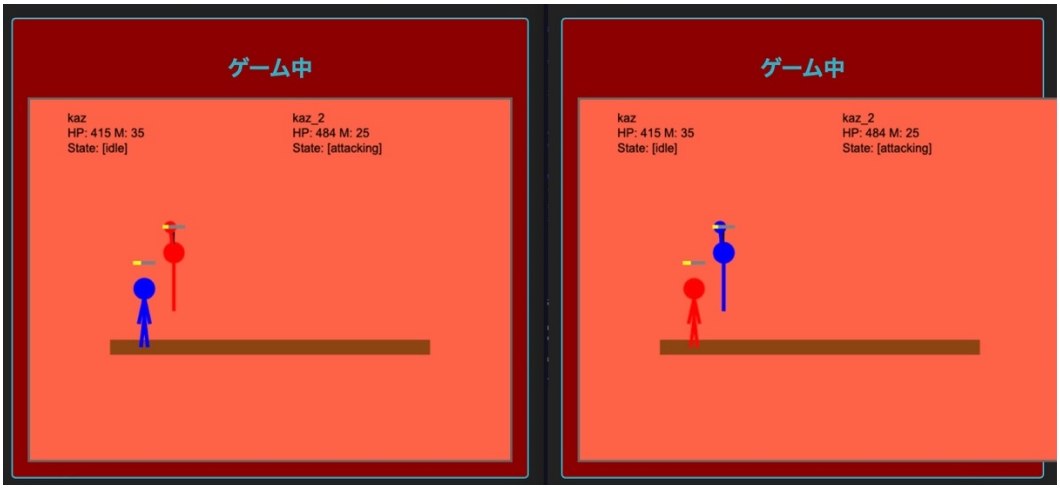
[ゲーム開始]



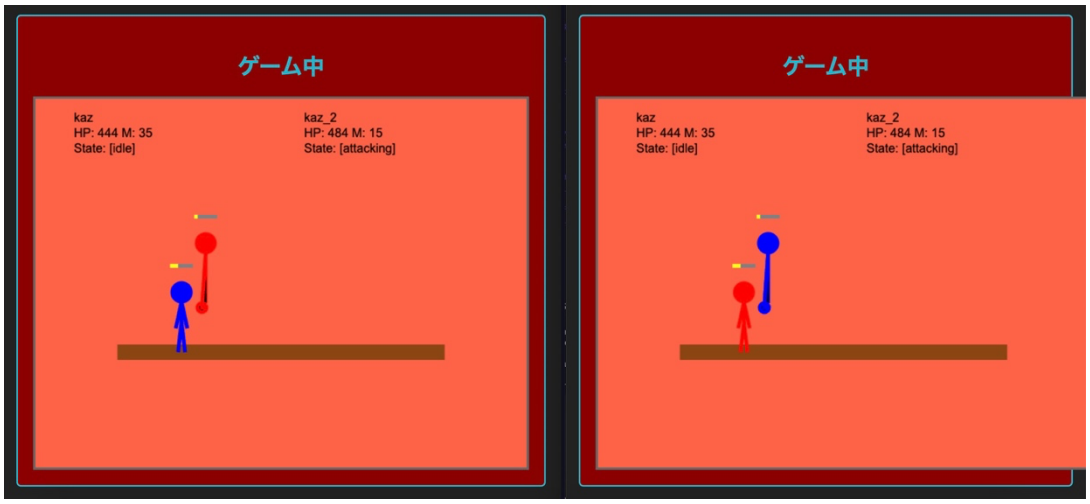
[攻撃(横)]※なぜかジャンプ中しかモーションが出ない



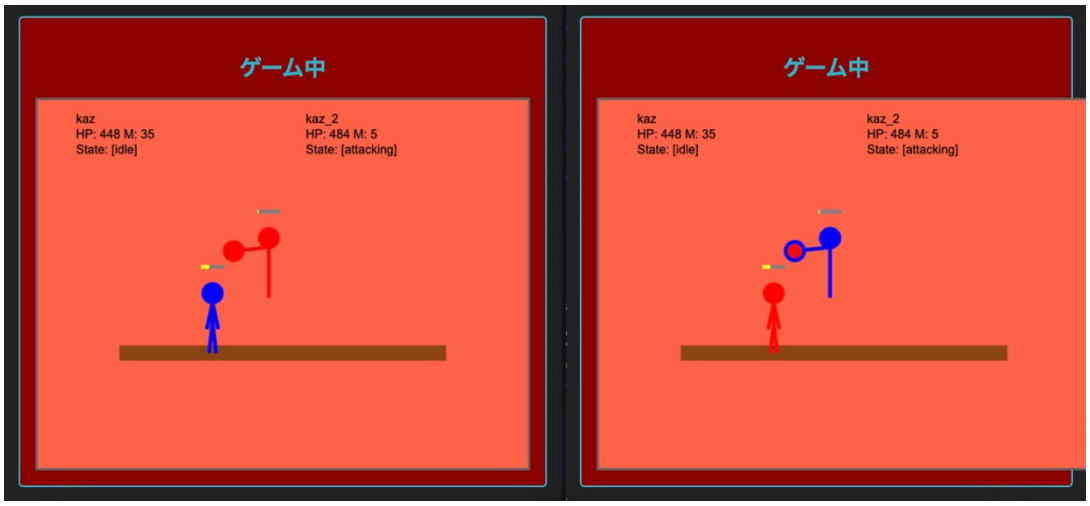
[攻撃(上)]



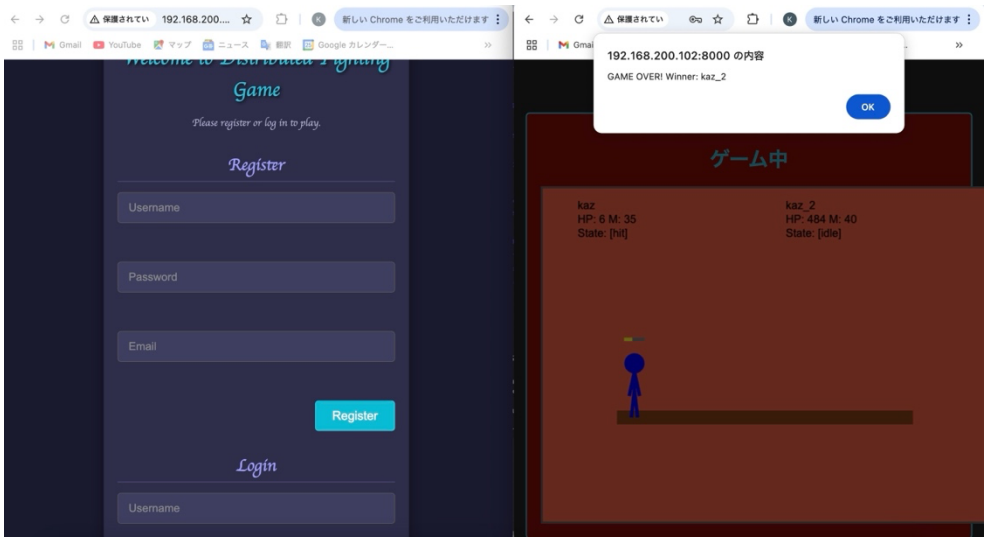
[攻撃(下)]



[攻撃(必殺技)]※M が 100 になったら発動可能



[ゲームオーバー]



2.2.4. データサービス (data_service)

目的: 全てのサービスが依存するデータの共通言語と構造を定義する。

役割:

1. users テーブル

認証サービス (`auth_match_service`) がユーザーのログイン情報を格納するために使用。`username` と `password_hash` を照合することで、ユーザーの本人確認を行う。status フィールド(pending, approved, rejected)により、管理者によるユーザーの利用可否を一元管理する。

2. games_sessions テーブル

マッチングが成立し、ゲームが開始された際に、そのセッションごとの状態を記録するために使用する。server_ip と server_port を保持することで、マッチングサービスがクライアントにどのゲームセッションサーバーに接続すべきかを指示するための情報源となる。

[server_config/db_init_scripts/init.sql]※PostgreSQL データベース(game_db_postgres コンテナ)が初めて起動する際に実行される初期化スクリプト

```sql
-- ユーザーテーブル (auth_match_service が使用)
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    status VARCHAR(50) DEFAULT 'pending' NOT NULL -- pending, approved,
rejected
);
-- ゲームセッションテーブル (game_session_service が使用)
CREATE TABLE IF NOT EXISTS game_sessions (
    session_id VARCHAR(255) PRIMARY KEY,
    mode VARCHAR(50) NOT NULL,
    players TEXT[] NOT NULL, -- プレイヤーID の配列
    server_ip VARCHAR(255) NOT NULL,
    server_port INT NOT NULL,
    status VARCHAR(50) DEFAULT 'active' NOT NULL, -- active, completed,
cancelled
    start_time TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    end_time TIMESTAMP WITH TIME ZONE
);
```

3. Nginx の役割とネットワーク制御

3.1. Nginx の三つの主要な役割

本システムのゲートウェイとして機能する Nginx（`nginx_s1`）は、以下の役割を担い、システムの可用性とセキュリティに貢献している。

役割	説明
リバースプロキシ	クライアントからの API リクエスト（例: `/api/game/match`）を Docker ネットワーク内部の `auth_match_service` の正しいパス（`/match`）に**パス置換して転送**する。これにより、クライアントは内部サービス構造を意識せずに済む。
静的ファイル配信	ルートパス（`/`）へのアクセスに対し、クライアントアプリケーションの HTML, CSS, JavaScript ファイルを配信する。
ロードバランサー	複数のバックエンドサービス（特に将来的に増える `game_session_service`）へのトラフィック分散基盤を提供する。

3.2. 簡易セキュリティツールとしての機能（`limit_req_zone`）

limit_req_zone の役割：Nginx を通過するトラフィックに対して、トラフィック制御（Traffic Policing）を行い、システム全体の安定性を確保する。

・動作: Nginx は、各 IP アドレスからの API リクエスト頻度を監視し、`rate=5r/s`（1 秒あたり 5 リクエスト）などの制限を超える通信を検知する。

・セキュリティへの寄与: これは、サーバーの入り口で不正な過剰なリクエスト（サービス拒否攻撃 - DoS/DDoS）を検知し、制限を超えた通信を `429 Too Many Requests` で遮断する。Nginx がリクエストの過剰性をチェックし、バックエンドサービスの負荷を保護する簡易的なセキュリティツールとして機能している。

3.3. テスト結果

コマンド：for i in {1..10}; do curl -s -o /dev/null -w "%{http_code}¥n" -X POST -H "Content-Type: application/json" -d '{"user_id":"test","mode":"ffa_2p"}' http://192.168.200.102:8000/api/game/match; done

実行結果(スペースの関係上、縦の出力を横にした)：

202 202 202 202 202 202 503 503 503 503

評価：

429 Too Many Requests を返す前に、503 Service Unavailable を返した。これは、Nginx のレート制限機能が作動する前に、バックエンドの auth_match_service がリクエストの過剰な集中により応答不能（一時的なフリーズやクラッシュ）に陥り、Nginx がその接続エラーを 503 としてクライアントに返したことを示している。結論として、レート制限（limit_req_zone）が、バックエンドの過負荷による不安定化を防ぐための予防的なセキュリティ機能として必要不可欠であることが、この 503 エラーの発生によって実証された。つまり、このサーバーに対しては 1 秒あたりのリクエストを少なくする必要があると分かった。

4. ゲーム開始までの流れ(再掲)

4.1. ユーザー認証の動作（管理者承認システム）

1. クライアントから新規登録を申請すると、リクエストは `auth_match_service` を経由し、`approval_service` にユーザー情報が送信される。
2. `auth_match_service` はユーザー名とパスワードをデータベースの `users` テーブルに仮登録し、`status` は `pending` となる。
3. 管理者が `approval_service` の管理画面でユーザーを `approved` に設定後、初めてそのユーザーはログインが可能になる。

4.2. マッチングシステムとゲームセッションの起動

1. ログイン後、クライアントから `auth_match_service` の `/api/game/match` エンドポイントへ POST リクエストが送信される。
2. `auth_match_service` 内の `matcher.py` がロジックを実行し、必要な人数（2 人）が揃うまでキューで待機させる（ネットワークログ: `202 Accepted`）。
3. マッチング成立後、`matcher.py` はサーバー1 の IP とポート情報をクライアントに返す（ネットワークログ: `200 OK`）。
4. クライアントは受け取った情報に基づき、WebSocket で `game_session_service` に接続し、ゲームが開始される。

6. まとめと今後の展望

- **成果:** 認証、マッチング、ゲームロジック、データ管理を独立させた 4 つの主要なサービスと Nginx を組み合わせた格闘ゲームを実現させた。
- **苦労した点**:コンテナのビルドエラーやパスの間違いによる接続エラーへの対処、多機能な格闘ゲームの実現が特に苦労した。
- **今後の課題:** ゲームセッションサービスにおけるジャンプ中の攻撃バグ修正、データサービスとの連携強化（勝敗記録の永続化）、そして WebSocket 通信の低遅延化などが課題である。また、現在はゲームがサーバー1 のみで行われているが、将来的には負荷の低いサーバーにゲームセッションを割り振れるようになりたい。